Labo4: Optimization techniques, Coalesced memory access and device (GPU) memory types

Arthur Valentin, Joeri Sevens

November 27, 2024

Overview

Deze labo richt zich op het verbeteren van prestaties binnen GPU-programmering door te onderzoeken hoe geheugenorganisatie en -toegang de rekentijd beïnvloeden. De focus ligt op het ontwerpen en evalueren van kernels die optimaal gebruikmaken van verschillende geheugenstructuren en toegangspatronen.

In deel 1 wordt een kernel ontwikkeld om de rode kleurcomponent van een afbeelding te inverteren. Hier wordt het verschil in uitvoeringstijd geanalyseerd tussen gecoalescede en niet-gecoalescede geheugenaccesses.

In deel 2 wordt een matrixvermenigvuldiging geïmplementeerd met verschillende geheugenstructuren:

- Alleen globaal geheugen.
- Een combinatie van globaal en gedeeld geheugen.
- Een combinatie van globaal en constant geheugen.

De verwerkingstijd wordt geëvalueerd door de gridconfiguratie en/of de invoergrootte aan te passen. Dit biedt inzicht in hoe het gebruik van GPU-geheugenniveaus de prestaties van computationeel intensieve kernels kan optimaliseren.

Methodology

Deel 1

Voor dit onderdeel hebben we twee kernels ontwikkeld: een uncoalesced kernel en een coalesced kernel. Bij de uncoalesced kernel verwerken we de waarden in de array van de afbeelding in de volgorde waarin ze worden aangeleverd, en bewerken we rechtstreeks elk derde element. Bij de coalesced kernel herschikken we de array eerst, zodat alle rode elementen aan het begin staan, gevolgd door alle groene en daarna de blauwe. Vervolgens bewerken we alleen het eerste derde deel van de array. Na de bewerking herordenen we de array weer naar de oorspronkelijke RGB-volgorde. Deze aanpak zorgt ervoor dat de rode elementen tijdens de verwerking aaneengesloten in het geheugen liggen, waardoor geheugenblokken optimaal worden benut.

Deel 2

Voor deel 2 van de opdracht hebben we een code geschreven die de verschillende methoden voor matrixvermenigvuldiging vergelijkt. De matMulGlobal kernel gebruikt uitsluitend globaal geheugen, waarbij elke thread een rij en kolom van de matrices verwerkt. De matMulShared kernel optimaliseert de berekeningen door blokken van de matrices in shared memory te laden, wat de toegangstijd tot het geheugen vermindert. De matMulConstant kernel maakt gebruik van constant geheugen, waarin matrix B wordt opgeslagen om de prestaties te verbeteren bij veelvuldig lezen.

In de code worden matrices geïnitialiseerd, gekopieerd naar de GPU en vermenigvuldigd met elk van de methoden. Vervolgens worden de uitvoeringstijden geëxporteerd naar een CSV-bestand.

Conclusion

Deel 1: What is the performance difference between the coalesced and uncoalesced kernel?

Coalesced geheugentoegang is efficiënter dan uncoalesced geheugentoegang. Bijvoorbeeld, bij een inputarray van 9.000.000 elementen, had de coalesced kernel gemiddeld slechts 84 ms nodig, terwijl de uncoalesced kernel 102 ms in beslag nam. Deze snelheidswinst ontstaat doordat de coalesced kernel

slechts een derde van het geheugen hoeft te lezen in vergelijking met de uncoalesced kernel, waarbij het geheugen altijd in blokken wordt gelezen.

Deel 1: How does the performance difference change dependent on input and/or grid size?

Als de inputgrootte en/of gridgrootte toenemen, zal het prestatverschil ook groter worden. Het percentageverschil blijft echter relatief constant, ongeveer 20% in ons geval.

Deel 2: How much time does memory migration take (CPU to GPU global; GPU global to GPU shared)?

In de onderstaande grafiek, zie figuur 1, zijn de resultaten weergegeven voor matrices die continu worden vergroot, en de tijd die elke geheugenoverdracht in beslag neemt. We kunnen duidelijk concluderen dat de migratie van CPU naar GPU global trager is dan de migratie van GPU global naar GPU shared.

Deel 2: How do the execution times of matrix multiplication compare when employing the three different memory types?

Zie antwoord hier onder.

Deel 2: How do the times scale with input and/or grid size?

Alle geheugentransacties nemen toe naarmate de input- en/of gridgrootte groter wordt, maar niet in dezelfde mate. Zie figuur 1. Zo zien we bijvoorbeeld dat wanneer alleen global memory wordt gebruikt, de uitvoeringstijd sterk toeneemt (bijna exponentieel) naarmate de inputgrootte stijgt. Wanneer de kernel zowel shared als global memory gebruikt, kunnen de taken al sneller worden uitgevoerd. De snelste uitvoering word bereikt wanneer de kernel een combinatie van constant en global memory gebruikt.

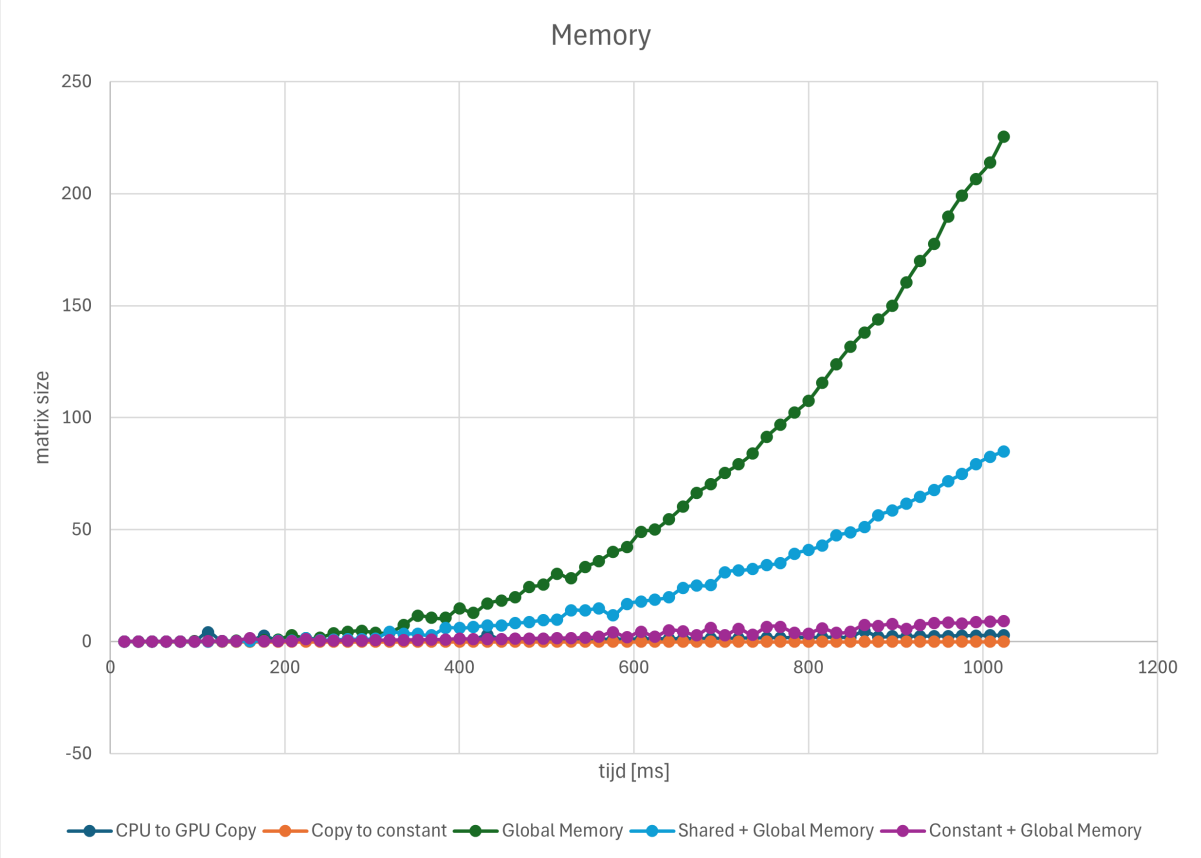


Figure 1: Uitvoeringstijd als functie van de grootte van vierkantsmatrices

Code Uncoalescend VS Coalescend

```cpp
#include <stdio.h>
#include <cuda_runtime.h>
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <fstream>      // For file operations
#include <cstdlib>      // For random number generation
#include <ctime>        // For seeding random number generation

__global__ void invert_red_uncoalesced(uint8_t* A, int length) {
    int i = (blockIdx.x * blockDim.x + threadIdx.x) * 3;
    if (i < length) {
        A[i] = 255 - A[i]; // Invert the red component
    }
}

__global__ void invert_red_coalesced(uint8_t* A, int derde_length) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < derde_length) {
        A[i] = 255 - A[i]; // Invert the red component
    }
}

void RGB_sorter(uint8_t* input, uint8_t* output, int length) {
    int red_idx = 0, green_idx = length / 3, blue_idx = 2 * length / 3;
    for (int i = 0; i < length; i += 3) {
        output[red_idx++] = input[i];
        output[green_idx++] = input[i + 1];
        output[blue_idx++] = input[i + 2];
    }
}

void revert_RGB(uint8_t* input, uint8_t* output, int length) {
    int red_idx = 0, green_idx = length / 3, blue_idx = 2 * length / 3;
    for (int i = 0; i < length; i += 3) {
        output[i] = input[red_idx++];
        output[i + 1] = input[green_idx++];
        output[i + 2] = input[blue_idx++];
    }
}

int main() {
    srand(time(0));

    std::ofstream outFile("coalescend_vs_uncoalescend_100_groot_derde_length_3m.csv
        ");
    outFile << "Iteration,CoalescedTime(ms),UncoalescedTime(ms)\n";

    int length = 300000000;
    uint8_t* image_array = (uint8_t*)malloc(length * sizeof(uint8_t));
    for (int i = 0; i < length; i++) {
        image_array[i] = static_cast<uint8_t>(rand() % 256);
        }

    uint8_t* sorted_image_array = (uint8_t*)malloc(length * sizeof(uint8_t));
    uint8_t *d_image_array, *d_sorted_image_array;

    RGB_sorter(image_array, sorted_image_array, length);

    for (int iter = 1; iter <= 10; iter++) {

        cudaMalloc(&d_sorted_image_array, length * sizeof(uint8_t));
```

```
61          cudaMemcpy(d_sorted_image_array, sorted_image_array, length * sizeof(
                uint8_t), cudaMemcpyHostToDevice);
62
63          int threadsPerBlock = 1024;
64          int blocksPerGrid = ((length/3) + threadsPerBlock - 1) / threadsPerBlock;
65
66          // Timing for coalesced kernel
67          float coalesced_ms;
68
69          cudaEvent_t start_coalesced, stop_coalesced;
70          cudaEventCreate(&start_coalesced);
71          cudaEventCreate(&stop_coalesced);
72          cudaEventRecord(start_coalesced);
73
74          invert_red_coalesced<<<blocksPerGrid, threadsPerBlock>>>(
                d_sorted_image_array, (length/3));
75          cudaDeviceSynchronize();
76
77          cudaEventRecord(stop_coalesced);
78          cudaEventSynchronize(stop_coalesced);
79          cudaEventElapsedTime(&coalesced_ms, start_coalesced, stop_coalesced);
80          cudaEventDestroy(start_coalesced);
81          cudaEventDestroy(stop_coalesced);
82
83          cudaFree(d_sorted_image_array);
84
85          // Timing for uncoalesced kernel
86          cudaMalloc(&d_image_array, length * sizeof(uint8_t));
87          cudaMemcpy(d_image_array, image_array, length * sizeof(uint8_t),
                cudaMemcpyHostToDevice);
88          float uncoalesced_ms;
89
90          cudaEvent_t start_uncoalesced, stop_uncoalesced;
91          cudaEventCreate(&start_uncoalesced);
92          cudaEventCreate(&stop_uncoalesced);
93          cudaEventRecord(start_uncoalesced);
94
95          invert_red_uncoalesced<<<blocksPerGrid, threadsPerBlock>>>(d_image_array,
                length);
96          cudaDeviceSynchronize();
97
98          cudaEventRecord(stop_uncoalesced);
99          cudaEventSynchronize(stop_uncoalesced);
100         cudaEventElapsedTime(&uncoalesced_ms, start_uncoalesced, stop_uncoalesced);
101         cudaEventDestroy(start_uncoalesced);
102         cudaEventDestroy(stop_uncoalesced);
103
104         outFile << iter << "," << coalesced_ms << "," << uncoalesced_ms << "\n";
105
106         cudaFree(d_image_array);
107
108     }
109
110     free(image_array);
111     free(sorted_image_array);
112
113     outFile.close();
114
115     return 0;
116 }
```

Memory

```cpp
#include <iostream>
#include <cstdio>
#include <cstdlib>
#include <cuda_runtime.h>
#include <fstream>      // For file operations

#define TILE_SIZE 16

// Kernel to multiply matrices using global memory only
__global__ void matMulGlobal(float* A, float* B, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0;

    if (row < N && col < N) {
        for (int k = 0; k < N; k++) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

// Kernel to multiply matrices using shared memory
__global__ void matMulShared(float* A, float* B, float* C, int N) {
    __shared__ float sA[TILE_SIZE][TILE_SIZE];
    __shared__ float sB[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0;

    for (int tile = 0; tile < (N + TILE_SIZE - 1) / TILE_SIZE; ++tile) {
        if (row < N && (tile * TILE_SIZE + threadIdx.x) < N) {
            sA[threadIdx.y][threadIdx.x] = A[row * N + tile * TILE_SIZE + threadIdx
                .x]; // Load from global to shared
        } else {
            sA[threadIdx.y][threadIdx.x] = 0.0;
        }
        if (col < N && (tile * TILE_SIZE + threadIdx.y) < N) {
            sB[threadIdx.y][threadIdx.x] = B[(tile * TILE_SIZE + threadIdx.y) * N +
                 col]; // Load from global to shared
        } else {
            sB[threadIdx.y][threadIdx.x] = 0.0;
        }
        __syncthreads();

        for (int k = 0; k < TILE_SIZE; ++k) {
            sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
        }
        __syncthreads();
    }

    if (row < N && col < N) {
        C[row * N + col] = sum;
    }
}

// Kernel to multiply matrices using constant memory (smaller size to avoid limit)
__constant__ float constB[TILE_SIZE * TILE_SIZE]; // Matrix B in constant memory

__global__ void matMulConstant(float* A, float* C, int N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
```

```
61      int col = blockIdx.x * blockDim.x + threadIdx.x;
62      float sum = 0;
63
64      if (row < N && col < N) {
65          for (int k = 0; k < TILE_SIZE; k++) {
66              sum += A[row * TILE_SIZE + k] * constB[k * TILE_SIZE + col]; // Use
                    constant memory for B
67          }
68          C[row * TILE_SIZE + col] = sum;
69      }
70  }
71
72  void initializeMatrix(float* mat, int N) {
73      for (int i = 0; i < N * N; i++) {
74          mat[i] = static_cast<float>(rand()) / RAND_MAX;
75      }
76  }
77
78  int main() {
79      std::ofstream outFile("matrix_multiplication_timing.csv");
80      outFile << "MatrixSize,MemcpyH2D(ms),MemcpyH2Constant(ms),GlobalMemory1(ms),
            GlobalMemory2(ms),SharedMemory1(ms),SharedMemory2(ms),ConstantMemory1(ms),
            ConstantMemory2(ms)\n";
81
82      for (int N = TILE_SIZE; N <= TILE_SIZE * 64; N += TILE_SIZE) { // Loop over
            increasing matrix sizes
83          size_t bytes = N * N * sizeof(float);
84
85          // Allocate memory on host
86          float* h_A = (float*)malloc(bytes);
87          float* h_B = (float*)malloc(bytes);
88          float* h_C = (float*)malloc(bytes);
89
90          // Initialize matrices
91          initializeMatrix(h_A, N);
92          initializeMatrix(h_B, N);
93
94          // Allocate memory on device
95          float* d_A, * d_B, * d_C;
96          cudaMalloc(&d_A, bytes);
97          cudaMalloc(&d_B, bytes);
98          cudaMalloc(&d_C, bytes);
99
100         // Copy matrices to device (CPU to GPU global)
101         cudaEvent_t start, stop;
102         cudaEventCreate(&start);
103         cudaEventCreate(&stop);
104         cudaEventRecord(start);
105         cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
106         cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
107         cudaEventRecord(stop);
108         cudaEventSynchronize(stop);
109         float memcpy_h2d_ms;
110         cudaEventElapsedTime(&memcpy_h2d_ms, start, stop);
111
112         // Copy B to constant memory (CPU to GPU constant)
113         cudaEventRecord(start);
114         cudaMemcpyToSymbol(constB, h_B, bytes);
115         cudaEventRecord(stop);
116         cudaEventSynchronize(stop);
117         float memcpy_h2constant_ms;
118         cudaEventElapsedTime(&memcpy_h2constant_ms, start, stop);
119
```

```
        dim3 threads(TILE_SIZE, TILE_SIZE);
        dim3 blocks((N + TILE_SIZE - 1) / TILE_SIZE, (N + TILE_SIZE - 1) /
            TILE_SIZE);

        // Timing for global memory kernel
        float global_ms1, global_ms2;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);
        matMulGlobal<<<blocks, threads>>>(d_A, d_B, d_C, N);
        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&global_ms1, start, stop);

        cudaEventRecord(start);
        matMulGlobal<<<blocks, threads>>>(d_A, d_B, d_C, N);
        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&global_ms2, start, stop);

        // Timing for shared memory kernel
        float shared_ms1, shared_ms2;

        cudaEventRecord(start);
        matMulShared<<<blocks, threads>>>(d_A, d_B, d_C, N);
        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&shared_ms1, start, stop);

        cudaEventRecord(start);
        matMulShared<<<blocks, threads>>>(d_A, d_B, d_C, N);
        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&shared_ms2, start, stop);

        // Timing for constant memory kernel
        float constant_ms1, constant_ms2;

        cudaEventRecord(start);
        matMulConstant<<<blocks, threads>>>(d_A, d_C, N);
        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&constant_ms1, start, stop);

        cudaEventRecord(start);
        matMulConstant<<<blocks, threads>>>(d_A, d_C, N);
        cudaDeviceSynchronize();
        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        cudaEventElapsedTime(&constant_ms2, start, stop);

        // Write results to CSV
        outFile << N << "," << memcpy_h2d_ms << "," << memcpy_h2constant_ms << ","
                << global_ms1 << "," << global_ms2 << ","
                << shared_ms1 << "," << shared_ms2 << ","
                << constant_ms1 << "," << constant_ms2 << "\n";

```

```
        // Clean up
        cudaFree(d_A);
        cudaFree(d_B);
        cudaFree(d_C);
        free(h_A);
        free(h_B);
        free(h_C);
    }

    outFile.close();

    return 0;
}
```